memakai 0cx0 (oven controll4ed crystal osillator), disana ada memakai 1 pps , oct clock dll. ini untuk clock synch antara satelite, ini kami belum menangkap jelas tapi prof lim bilang ini additional diluar oisl tapi ini bisa saja jadi pertimbangan, namun kita juga harus tau maksudnya apa gitu (coba cek apa ini dan kaitannya d;ll). case ini muncul saat dr lim menanyakan bagaimana kita membuat timestamp, kita perlu menejalskan juga bagaimana kita membuat ts dan perhitungan dll pada header atau di mgmt. coba telusuri ini

Response: Pembahasan yang disampaikan oleh Dr. Lim mengenai **OCXO (*Oven-Controlled Crystal Oscillator*)**, **1 PPS (*One Pulse Per Second*)**, **OCT (*Optical Communications Terminal*) Clock**, dan **sinkronisasi jam antar-satelit (*Inter-Satellite Clock Synchronization*)** berkaitan langsung dengan bagaimana sistem satelit di dunia nyata membangkitkan sinyal waktu (*timestamp*) yang masuk ke port input modul **`ethernet_fso_framer.v`** dan **`mgmt_telemetry_generator.v`** pada desain FPGA kita.

Berikut adalah penjelasan menyeluruh mengenai arti dari istilah-istilah tersebut, keterkaitannya dengan standar **SDA OISL v2.1.2**, serta bagaimana kode FPGA kita saat ini membuat dan menghitung *timestamp* pada **Header (16 Byte)** maupun pada **Frame MGMT (1.912 Byte)**.

1. Apa Itu OCXO, 1 PPS, dan OCT Clock? (Mengapa Disebut "Additional di Luar OISL" oleh Dr. Lim?)

Di dalam arsitektur satelit LEO (*Low Earth Orbit*), sistem komunikasi optik dibagi menjadi dua bagian: **(A) Blok Modem/Framer OISL** (yang sedang kita bangun di dalam FPGA) dan **(B) Unit Referensi Waktu Satelit (*Spacecraft PNT / Timekeeping Unit*)** yang berada di luar blok modem OISL.

Berikut adalah fungsi dari ketiga komponen waktu yang disebutkan oleh Dr. Lim:

Komponen / Istilah	Kepanjangan	Fungsi Fisik & Perannya dalam Sinkronisasi Satelit
1. OCXO	*Oven-Controlled Crystal Oscillator*	Osilator kristal presisi tinggi yang dibungkus di dalam ruang pemanas (*oven*) bersuhu konstan. Di luar angkasa, suhu satelit berubah drastis (dari $-100^{\circ}\mathrm{C}$ hingga $+100^{\circ}\mathrm{C}$). Kristal biasa (*XO* seperti kristal 200 MHz di board FPGA kita) akan mengalami pergeseran frekuensi (*frequency drift*) sekitar $20–50\ \mathrm{ppm}$ ($\pm 20–50\ \mu\mathrm{s}$ per detik). Sebaliknya, **OCXO** menjaga stabilitas frekuensi hingga $10^{-10} - 10^{-12}$ **(akurasi sub-nanodetik)** sehingga perhitungan jarak cahaya tidak meleset.

Komponen / Istilah	Kepanjangan	Fungsi Fisik & Perannya dalam Sinkronisasi Satelit
2. 1 PPS	*One Pulse Per Second*	Sinyal denyut listrik yang naik tepat **1 kali setiap 1 detik** (biasanya berasal dari penerima GPS/GNSS di satelit atau jam atom/OCXO utama). Di dalam sistem kita, sinyal **1 PPS** inilah yang menjadi sumber fisik bagi port **`mgmt_trigger_1hz`** untuk dua tugas: **(1)** mereset pencacah pikodetik (`0 .. 999.999.999.999 ps`) kembali ke `0` sambil menambah detik `tod_seconds_in` (+1), dan **(2)** memicu pengiriman **Frame MGMT/PNT** tepat 1 kali per detik!
3. OCT Clock	*Optical Communications Terminal Clock*	Sinyal *clock* sistem lokal di dalam terminal optik yang didisiplinkan oleh **OCXO + 1 PPS** (*1 PPS-disciplined OCXO*). Clock inilah yang menggerakkan **Pencacah Waktu Bebas (*Free-Running Picosecond Counter*)** untuk menghasilkan angka waktu 40-bit pikodetik (`tx_phy_ts_picosec` dan `rx_phy_ts_picosec`) yang disuplai ke FPGA kita.

Mengapa Dr. Lim Mengatakan Ini "Additional di Luar OISL, Tapi Perlu Dipertimbangkan"?

- Menurut dokumen standar **SDA OISL v2.1.2 (Seksi 2.6.3.3.4 & Seksi 2.6.3.5.2.3)**, tugas wajib dari modem OISL di FPGA adalah **mencatat (*latching*), mengompensasi latensi pipa, dan mengemas nilai *timestamp* ke dalam Frame Header serta Frame MGMT**.
- Adapun perangkat keras fisik **OCXO dan penerima 1 PPS** merupakan perangkat keras eksternal di luar kode RTL modem OISL.
- Namun, jika pada tahap integrasi perangkat keras nanti ETRI ingin menyambungkan generator **1 PPS + OCXO eksternal** ke papan **ALINX AXKU062** kita, papan kita **sudah memiliki jalur masuk khusus**, yaitu **2 konektor koaksial SMA eksternal (`SMA_CLKIN_P` di pin `H12` dan `SMA_CLKIN_N` di pin `G12` pada BANK 66, level 1,8V)** sebagaimana tercantum pada **Bagian 3.8 (Halaman 50) AXKU062 User Manual**!

2. Bagaimana Sinkronisasi Jam Antar-Satelit (*Clock Synch*) & Pengukuran Jarak (*Ranging*) Bekerja?

Alasan utama mengapa kita mengirimkan *timestamp* di dalam **Header** dan **Frame MGMT**

adalah untuk menjalankan algoritma **TWTT (*Two-Way Time Transfer*)**.

Misalkan **Satelit A** dan **Satelit B** terpisah sejauh **5.000 km**, dan jam lokal di Satelit B memiliki selisih waktu (*clock offset* $\Delta\theta$) terhadap Satelit A:

1. $\mathbf{T_1}$ **(TX_TS Satelit A):** Satelit A memancarkan frame FSO dan mencatat waktu tepat ketika bit pertama *Preamble* keluar dari laser TX: T_1.
2. $\mathbf{T_2}$ **(`RX_PHY_Timestamp` Satelit B):** Satelit B mendeteksi bit pertama *Preamble* tersebut tiba di fotodetektor RX dan mencatat waktu lokalnya: T_2.
3. $\mathbf{T_3}$ **(`TX_PHY_Timestamp` Satelit B):** Satelit B memancarkan frame balasan pada waktu lokalnya T_3, sambil memasukkan nilai (T_2, T_3) **ke dalam muatan Frame MGMT (Tabel 2-14)** untuk dikirim kembali ke Satelit A.
4. $\mathbf{T_4}$ **(`RX_PHY_Timestamp` Satelit A):** Satelit A mencatat waktu kedatangan frame balasan tersebut pada jam lokalnya: T_4.

Dengan memiliki keempat angka waktu (T_1, T_2, T_3, T_4) yang dikumpulkan dari Header dan Frame MGMT tersebut, Satelit A langsung mendapatkan **dua besaran fisika sekaligus**:

- **1. Waktu Tempuh Cahaya (τ / *Time-of-Flight*) & Jarak Antar-Satelit (R / `range_meters`):**

  $$\tau = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}, \qquad R = c \times \tau$$

  (Perhatikan bahwa karena $(T_4 - T_1)$ diukur oleh jam Satelit A dan $(T_3 - T_2)$ diukur oleh jam Satelit B, selisih offset jam kedua satelit otomatis saling menghilangkan!)

- **2. Selisih Jam Antar-Satelit ($\Delta\theta$ untuk *Inter-Satellite Clock Synchronization*):**

  $$\Delta\theta = \frac{(T_2 - T_1) - (T_4 - T_3)}{2}$$

  Nilai $\Delta\theta$ inilah yang dipakai oleh satelit untuk menyinkronkan jam **OCT Clock / OCXO** mereka hingga akurasi pikodetik/sub-nanodetik!

3. Bagaimana Kode FPGA Kita Saat Ini Membuat & Menghitung Timestamp (Header vs. MGMT)

Saat Dr. Lim menanyakan bagaimana desain kita membuat dan menghitung *timestamp*, berikut adalah mekanisme eksak yang sudah berjalan di dalam kode RTL kita beserta rumus perhitungannya:

A. Pembuatan & Perhitungan Timestamp pada FSO Frame Header (16 Byte)

Di dalam **16-Byte Frame Header (Tabel 2-9 & Tabel 2-10)** yang dibentuk oleh **`ethernet_fso_framer.v`** dan **`header_generator.v`** (`fso_header_top.v`), terdapat **3 field waktu**:

1. `TX_TS [39:0]` (40-bit, terletak pada Byte `d5` s.d. `d9`, LSB-first):
    - **Mengapa lebarnya 40-bit?** Karena TX_TS menghitung waktu dalam satuan **pikodetik (*picoseconds*,** $1\ \text{ps} = 10^{-12}\ \text{detik}$**)** di dalam rentang 1 detik ($0 \ldots 999.999.999.999\ \text{ps}$). Angka 10^{12} membutuhkan $\lceil \log_2(10^{12}) \rceil = \mathbf{40}\ \text{bit}$ (nilai maksimum $2^{40} - 1 = 1.099.511.627.775\ \text{ps} \approx 1,0995\ \text{detik}$).
    - Rumus Kompensasi Latensi Pipa Deterministik (`TX_PIPE_LATENCY_PS`) di `ethernet_fso_framer.v`: Standar SDA OISL mewajibkan nilai TX_TS yang tertulis di dalam Header menunjukkan **momen fisik ketika bit pertama (MSB) dari *Preamble* 72-bit keluar dari transceiver optik GTH TX**. Padahal, nilai TX_TS tersebut sudah harus dimasukkan ke dalam Header **sebelum** Header dihitung CRC-16-nya, dienkode oleh $\text{RS}(32, 16)$, diacak oleh *Scrambler*, dienkode oleh $\text{LPC}(24, 16)$, dan melewati FIFO CDC 1 serta CDC 2! Karena seluruh jalur pipa TX kita memiliki jumlah siklus *clock* yang tetap (*deterministic latency*), pada **`ethernet_fso_framer.v` baris 28 & 110** kita menghitung prediksi waktu keluar fisik secara otomatis:

      $$\text{predicted_egress_tx_ts} = \text{tx_phy_ts_picosec} + \text{TX_PIPE_LATENCY_P}$$

      dengan rincian konstanta `TX_PIPE_LATENCY_PS = 40'd2_313_200` (**`2.313.200 ps = 2.313,2 ns`**):
        - Latensi serialisasi Header & CRC-16 di `fso_header_top` (`312,5 MHz`): $16\ \text{siklus} \times 3,2\ \text{ns} = 51,2\ \text{ns}$.
        - Latensi penjadwalan CDC 1 (`x_gth_logic.v`) + enkoding $\text{RS}(32, 16)$ & $\text{RS}(255, 239)$ + `FEC_MUX` + `scrambler` + `lpc_top` + `mux_gearbox_128` (`200 MHz`): $446\ \text{siklus} \times 5,0\ \text{ns} = 2.230,0\ \text{ns}$.
        - Latensi pipa baca CDC 2 (`x_logic_gth.v`) + `gth64b66b_tx` (`312,5 MHz`): $10\ \text{siklus} \times 3,2\ \text{ns} = 32,0\ \text{ns}$.
        - **Total Latensi Pipa TX:** $51,2\ \text{ns} + 2.230,0\ \text{ns} + 32,0\ \text{ns} = \mathbf{2.313,2}\ \text{ns} = \mathbf{2.313.200}\ \text{ps}$!
2. `TOD_SECONDS [5:0]` (6-bit, terletak pada Byte `d10[5:0]`, dengan bit `[7:6] = 2'b11` Reserved):
    - Menyimpan hitungan detik *Time-of-Day* ($0 \ldots 59$ detik) yang bertambah +1

setiap kali denyut **1 PPS** menyala.

3. TS_APPLIES `[2:0]` (3-bit, terletak pada Byte `d11[6:4]`, dengan bit `[7] = 1'b1` Reserved):
    - Menandakan TX_TS tersebut milik frame ke berapa: nilai **`3'b000` (0)** berarti berlaku untuk **frame saat ini (*current frame*)**, sedangkan `1..7` berarti berlaku untuk 1..7 frame sebelumnya (jika sistem tidak menggunakan pre-kompensasi latensi pipa). Karena sistem kita sudah menambahkan pre-kompensasi TX_PIPE_LATENCY_PS, maka saat `ts_valid == 1'b1`, `active_ts_applies` bernilai **`3'b000`** (`current frame`).

B. Pembuatan & Perhitungan Timestamp pada Frame MGMT/PNT (1.912 Byte — Tabel 2-14)

Setiap 1 detik (ketika `mgmt_trigger_1hz` / **1 PPS** menyala), `arq_tx_controller.v` menjadwalkan pengiriman **Frame MGMT (`FRAME_TYPE = 2'b10`)** dengan prioritas ke-2. Modul **`mgmt_telemetry_generator.v`** mengemas **229 bit data waktu dan jarak (Tabel 2-14)** ke dalam 8 kata 32-bit pertama (`256 bit` = `32 Byte`), diikuti `469 kata` (`1.876 Byte`) *Zero Padding* + `1 kata` (`4 Byte`) CRC-32 = **478 kata (1.912 Byte)**:

Posisi Bit (Tabel 2-14)	Nama Sinyal di RTL Kita	Lebar Bit	Nilai Simulasi 5.000 km Kita & Cara Perhitu
Bit `0..5`	`rx_phy_ts_sec`	6-bit	**`6'd15`** (Detik ke-15 saat frame diterima di RX)
Bit `6..45`	`rx_phy_ts_picosec`	40-bit	**`40'd16_678_204_759`** ps. Dihitung dari rum untuk jarak $R = 5.000.000\ \mathrm{m}$: $\tau = \frac{5.000.000\ \mathrm{m}}{299.792.458\ \mathrm{m/s}} = 0,016678204759$
Bit `46..51`	`tx_phy_ts_sec`	6-bit	**`6'd15`** (Detik ke-15 saat frame dipancarkan d
Bit `52..91`	`tx_phy_ts_picosec`	40-bit	**`40'd0`** ps (Dipancarkan tepat pada awal detik
Bit `92..155`	`range_meters`	64-bit	**`64'h4153_12D0_0000_0000`**. Ini adalah repr **Precision Floating-Point (Float64)** untuk jar $5.000\ \mathrm{km}$**)**:

Posisi Bit (Tabel 2-14)	Nama Sinyal di RTL Kita	Lebar Bit	Nilai Simulasi 5.000 km Kita & Cara Perhitu
			• Sign = `0` (positif) • Exponent = `1023 + 22 = 1045` (`11'h415`) • Mantissa = $\frac{5.000.000}{2^{22}} - 1 = 0,19209289550$ $52'h3_12D0_0000_0000$ • Digabung menjadi: **`0x4153_12D0_0000_00`**
Bit 156..187	`range_ts_sec`	32-bit	**`32'd1_476_000_000`** (Detik Epoch referensi dihitung).
Bit 188..227	`range_ts_picosec`	40-bit	**`40'd0`** ps.
Bit 228	`ts_valid`	1-bit	**`1'b1`** (Menandakan bahwa pengukuran *rang*
Bit 229..255	*Zero Padding*	27-bit	**`27'd0`** (Menggenapkan 8 kata 32-bit = 256 bi

4. Bagaimana Jika Kita Ingin Menambahkan Counter Waktu Bebas (*Free-Running OCT Clock & 1 PPS Counter*) di FPGA?

Di dalam `SDAOISL10Gto10G_top.v` saat ini, port waktu pada `sda_encoder_inst` diisi dengan konstanta skenario uji jarak 5.000 km (`tx_phy_ts_picosec = 40'd0`, `rx_phy_ts_picosec = 40'd16678204759`) agar nilai di *waveform* mudah diverifikasi secara statis.

Namun, apabila Dr. Lim menanyakan: *"Bagaimana cara FPGA membangkitkan `tx_phy_ts_picosec`, `tod_seconds_in`, dan `mgmt_trigger_1hz` secara hidup (live counter) di perangkat keras?"*, cara kerjanya sangat sederhana dan presisi:

- Pada domain clock **`tx0_clk` (`312,5 MHz`)**, **1 siklus clock = tepat `3,2 ns` = `3.200` pikodetik (`ps`)**!

- Artinya, sebuah register pencacah 40-bit (`oct_picosec_cnt`) cukup **bertambah + `40'd3200` pada setiap siklus `tx0_clk`**!
- Ketika pencacah mencapai **`1.000.000.000.000 ps` (10^{12} ps = 1 detik, yaitu tepat setelah `312.500.000` siklus clock)**—atau ketika sinyal eksternal **1 PPS** masuk melalui pin **SMA (`H12 / G12`, Bank 66)** pada papan **ALINX AXKU062**—pencacah pikodetik di-reset kembali ke `0`, pencacah detik `tod_seconds_cnt` (`0..59`) bertambah +1, dan sinyal `mgmt_trigger_1hz` menyala selama 1 siklus untuk memicu pengiriman Frame MGMT!

Berikut adalah poin ringkas dalam bahasa Inggris yang siap Anda sampaikan kepada Dr. Lim saat menjelaskan bagian Timestamp ini:

> **1. Separation of External PNT Reference (OCXO + 1 PPS) and OISL Modem Timestamping:** "In a complete satellite optical terminal (OCT), the physical time reference—comprising a 1 PPS pulse from the spacecraft GNSS/time-transfer unit and a high-stability Oven-Controlled Crystal Oscillator (OCXO)—provides a disciplined local clock and 1 Hz epoch tick. On our ALINX AXKU062 hardware platform, external OCXO/1 PPS signals can be fed directly through the onboard differential SMA clock inputs (`Pin H12 / G12` in Bank 66), while the FPGA maintains a 40-bit picosecond counter incrementing by `3,200 ps` per `312.5 MHz` clock cycle (`0` to `999,999,999,999 ps`) and a 6-bit `TOD_SECONDS` counter (`0..59`)."
>
> **2. Frame Header Timestamp (TX_TS) with Deterministic Pipeline Pre-Compensation:** "Per SDA OISL Section 2.6.3.3.4, the 40-bit TX_TS in Header bytes `d5..d9` must reflect the exact physical instant the first (MSB) bit of the 72-bit Preamble exits the optical PHY. Because the header must be formatted before CRC-16 and RS(32,16) encoding, `ethernet_fso_framer.v` pre-compensates the live counter by adding the exact deterministic TX datapath latency (`TX_PIPE_LATENCY_PS = 2,313,200 ps` = `2,313.2 ns`) and sets `TS_APPLIES = 3'b000` (current frame)."
>
> **3. MGMT Frame Ranging Telemetry (Table 2-14):** "Triggered by the 1 Hz (1 PPS) tick, `mgmt_telemetry_generator.v` packs the 46-bit `RX_PHY_Timestamp`, 46-bit `TX_PHY_Timestamp`, 64-bit IEEE-754 double-precision `Range_Meters` (`0x4153_12D0_0000_0000` for 5,000 km with a `16,678,204,759 ps` one-way time-of-flight), and 72-bit `Range_TimeStamp` into bits `0..228` of the 1,912-Byte MGMT frame, enabling two-way time transfer (TWTT) and inter-satellite clock synchronization at the remote terminal."

Apakah Anda ingin kita menambahkan modul kecil **`oct_timekeeper_1pps.v`** (pencacah pikodetik *live* `+3200 ps/siklus` dengan input 1 PPS) ke dalam `SDAOISL10Gto10G_top.v` agar `tx_phy_ts_picosec` dan `tod_seconds_in` bergerak secara *real-time* di *waveform*, atau kita lanjutkan ke perakitan kode ARQ yang kita bahas sebelumnya?

Tambahkan modul oct_timekeeper_1pps.v ke SDAOISL10Gto10G_top.v agar tx_phy_ts_picosec dan tod_seconds_in berdetak secara live (+3200 ps/siklus)

Lanjutkan ke perakitan kode ARQ terbaru (sda_decoder.v, ethernet_fso_framer.v, arq_tx_controller.v, arq_ddr4_wrapper.v, dan error_inject.v)

Buatkan slide ilustrasi diagram blok OCXO + 1 PPS + Two-Way Time Transfer (T1, T2, T3, T4) untuk dipresentasikan ke Dr. Lim

---